

DevOps Complete Guide

Basic to Advanced: concepts, commands and code snippets

Linux | Git | Cloud (AWS, Azure, GCP) | Docker | Kubernetes | Terraform | Ansible | Jenkins | CI/CD | Helm | Monitoring | GitOps | DevSecOps

Each chapter goes from first principles to advanced usage, with comparison tables, ready-to-run snippets, common mistakes and interview-style questions.

Contents

1. Roadmap and DevOps Foundations	4
1.1 What is DevOps?	4
1.2 Suggested learning order	4
1.3 Core DevOps principles	4
2. Linux Essentials to Advanced	6
2.1 Why Linux for DevOps	6
2.2 Navigation and file management	6
2.3 Text processing power tools	7
2.4 Users, groups and permissions	8
2.5 Processes and system monitoring	8
2.6 Services with systemd	9
2.7 Package management	9
2.8 Networking commands	9
2.9 SSH and secure remote access	10
2.10 Disks, mounts and archives	10
2.11 Shell scripting	11
2.12 Troubleshooting playbook	12
3. Git and Version Control	13
3.1 Concepts	13
3.2 Daily workflow commands	13
3.3 Branching and merging	13
3.4 Undoing things safely	14
3.5 Remotes, tags and collaboration	14
3.6 Branching strategies	15
3.7 Advanced Git	15
4. Cloud Computing Fundamentals	16
4.1 What is cloud computing?	16
4.2 Global infrastructure	16
4.3 Core services mapped across providers	17
4.4 Compute in depth	17
4.5 Networking: VPC	18
4.6 Storage	18
4.7 Identity and Access Management (IAM)	18
4.8 Databases and load balancing	19
4.9 Well-Architected pillars	19
4.10 AWS CLI cheat sheet	19

4.11 Cost management tips	19
-------------------------------------	----

1. Roadmap and DevOps Foundations

1.1 What is DevOps?

DevOps is a culture plus a set of practices that unites software **development** and IT **operations** so that software is built, tested, released and operated quickly, reliably and repeatedly. The goal is short feedback loops: commit code, have it automatically built and tested, deployed to an environment, monitored, and improved.

The DevOps loop: Plan -> Code -> Build -> Test -> Release -> Deploy -> Operate -> Monitor -> (back to Plan). Every tool in this guide fits somewhere on this loop.

Stage	Purpose	Tools covered here
Code / version control	Track and collaborate on source	Git, GitHub, GitLab
Build / CI	Compile, test, package automatically	Jenkins, GitHub Actions, GitLab CI
Package	Reproducible runnable artifacts	Docker images, registries
Provision	Create servers, networks, clusters	Terraform, cloud consoles and CLIs
Configure	Install and configure software on servers	Ansible
Orchestrate	Run containers at scale	Kubernetes, Helm
Deploy / CD	Release automatically and safely	Jenkins, ArgoCD, rolling or canary releases
Monitor	Metrics, logs, alerts	Prometheus, Grafana, ELK, CloudWatch

1.2 Suggested learning order

- 1 **Linux and shell**: every other tool runs on Linux or is operated through a terminal.
- 2 **Git**: all code, scripts and infrastructure definitions live in Git.
- 3 **Cloud basics**: servers, networks, storage, IAM, so you know what you are automating.
- 4 **Docker**: package an app once, run anywhere.
- 5 **Kubernetes**: run many containers reliably.
- 6 **Terraform**: create the cloud infrastructure from code.
- 7 **Ansible**: configure machines from code.
- 8 **Jenkins and CI/CD**: glue everything into automated pipelines.
- 9 **Monitoring, GitOps, security**: operate and harden.

1.3 Core DevOps principles

- **Automation**: anything done twice should become a script or a pipeline.
- **Infrastructure as Code (IaC)**: servers and networks are described in files, versioned in Git, reviewed like application code.
- **Immutable infrastructure**: do not patch running servers; replace them with new, tested images.
- **Continuous Integration**: merge small changes often; each merge builds and tests automatically.
- **Continuous Delivery vs Deployment**: Delivery means every change is **ready** to release (manual approval gate); Deployment means every passing change is released automatically.
- **Shift left**: test and secure early (unit tests, linting, image scanning in CI).
- **Observability**: you can understand system state from metrics, logs and traces.

- **Idempotency:** running a task many times yields the same result as running it once (Ansible and Terraform are built on this idea).

2. Linux Essentials to Advanced

2.1 Why Linux for DevOps

Most servers, containers and cloud VMs run Linux. Docker containers share the Linux kernel; Kubernetes nodes are Linux; Jenkins agents, Ansible targets and Terraform-created VMs are usually Linux. Fluency with the shell is the single most valuable DevOps skill.

Key concepts

- **Kernel:** core of the OS, manages CPU, memory, devices, processes. **Shell** (bash, zsh, sh) is the command interpreter.
- **Everything is a file:** devices (`/dev/sda`), processes (`/proc/1234`), and sockets are exposed as files.
- **Distributions:** Debian/Ubuntu (apt, `.deb`) vs RHEL/CentOS/Rocky/Amazon Linux/Fedora (dnf/yum, `.rpm`) vs Alpine (apk, tiny, popular for containers).

Filesystem hierarchy

Path	Purpose
<code>/</code>	Root of the whole tree
<code>/home/user</code>	Personal files of each user; <code>~</code> is shorthand
<code>/root</code>	Home of the root (admin) user
<code>/etc</code>	System-wide configuration files (nginx, ssh, fstab, hosts)
<code>/var</code>	Variable data: logs (<code>/var/log</code>), spool, caches, databases
<code>/usr</code>	Installed programs and libraries (<code>/usr/bin</code> , <code>/usr/local</code>)
<code>/bin</code> , <code>/sbin</code>	Essential binaries (often symlinks into <code>/usr</code>)
<code>/opt</code>	Optional third-party software (for example Jenkins agents, custom apps)
<code>/tmp</code>	Temporary files, cleared on reboot
<code>/dev</code> , <code>/proc</code> , <code>/sys</code>	Devices and virtual kernel/process information
<code>/mnt</code> , <code>/media</code>	Mount points for extra disks

2.2 Navigation and file management

```

pwd                # print working directory
ls -lah            # long list, all files, human readable sizes
cd /var/log        # change directory; cd - (previous), cd ~ (home)
mkdir -p app/{src,logs,config} # create nested dirs, brace expansion
touch file.txt     # create empty file / update timestamp
cp -r src/ backup/ # copy recursively; cp -a keeps permissions/timestamps
mv old.txt new.txt # move or rename
rm file.txt        # delete file; rm -rf dir/ (DANGEROUS, no undo)
ln -s /opt/app/current /srv/app # symbolic link; ln (no -s) = hard link
tree -L 2          # directory tree two levels deep (install tree first)
file image.bin     # detect file type
stat file.txt      # size, inode, permissions, timestamps
du -sh *           # size of each item here; du -h --max-depth=1 /var | sort -h
df -h              # filesystem usage per mount

```

Viewing and searching files

```
cat file | less          # page through; less +F file follows like tail -f
head -n 20 file; tail -n 50 file; tail -f /var/log/syslog # follow live log
wc -l file              # count lines
find / -name "*.conf" -type f 2>/dev/null                # find by name
find /var/log -type f -mtime +7 -delete                  # delete files older than 7 days
find . -type f -size +100M                                # files larger than 100 MB
grep -rn "ERROR" /var/log/app/                            # recursive search, line numbers
grep -iE "warn|fail" app.log                             # case-insensitive, regex alternation
grep -v "^#" config.conf                                 # invert: hide comment lines
which python3; type ls; whereis nginx                     # locate commands
locate nginx.conf    # fast index based search (updatedb first)
```

2.3 Text processing power tools

These are used constantly for log analysis and quick automation.

```
# cut: pick columns; sort + uniq: count duplicates
cut -d: -f1,3 /etc/passwd          # user and uid
awk -F: '{print $1, $3}' /etc/passwd # same with awk
awk '$9 >= 500 {c++} END {print c}' access.log # count HTTP 5xx style lines (field 9)
awk '{sum+=$1} END {print sum/NR}' numbers.txt # average
sed 's/foo/bar/g' file              # substitute (stdout)
sed -i 's/8080/9090/' app.conf       # edit file in place
sed -n '10,20p' file                 # print lines 10 to 20
sort file | uniq -c | sort -nr | head # top repeated lines
cut -d' ' -f1 access.log | sort | uniq -c | sort -nr | head -5 # top 5 client IPs
tr 'a-z' 'A-Z' < in.txt              # translate characters
xargs                                # build commands from stdin
find . -name "*.log" | xargs gzip      # compress all logs
tee out.txt                           # write to file AND stdout
echo "line" | sudo tee -a /etc/hosts  # append to a root-owned file
jq '.items[].metadata.name' pods.json # JSON processing (very useful with cloud CLIs)
```

Pipes and redirection

Syntax	Meaning		
<code>`cmd1 \`</code>	<code>cmd2`</code>	Send stdout of cmd1 as stdin of cmd2	
<code>cmd > file / cmd >> file</code>	Overwrite / append stdout to file		
<code>cmd 2> err.txt</code>	Redirect stderr		
<code>cmd > all.txt 2>&1</code>	Redirect both stdout and stderr to the same file		
<code>cmd < file</code>	Feed file as stdin		
<code>cmd1 && cmd2</code>	Run cmd2 only if cmd1 succeeded		
<code>`cmd1 \`</code>	<code>\`</code>	<code>cmd2`</code>	Run cmd2 only if cmd1 failed
<code>cmd &</code>	Run in background		
<code>\$(cmd)</code>	Command substitution (capture output)		

2.4 Users, groups and permissions

Every file has an **owner**, a **group** and three permission sets (user, group, others), each with read (4), write (2), execute (1).

```
ls -l script.sh          # -rwxr-xr-- 1 alice dev 120 Jan 1 script.sh
chmod 755 script.sh      # rwx r-x r-x (owner full, others read+execute)
chmod u+x,g-w file       # symbolic form
chmod -R 750 /opt/app    # recursive
chown alice:dev file     # change owner and group; chown -R for directories
umask                    # default permission mask (022 -> new files 644, dirs 755)

# user management
sudo useradd -m -s /bin/bash -G sudo deploy # create user with home and shell
sudo passwd deploy
sudo usermod -aG docker deploy              # add to docker group (-a is essential!)
sudo userdel -r olduser
id deploy; groups deploy; whoami; w; last    # who is who / who is logged in
sudo -l                                     # what may I run with sudo
sudo visudo                                  # safely edit /etc/sudoers
```

Special bits: **setuid** (4xxx) runs a file as its owner; **setgid** (2xxx) inherits group on directories; **sticky bit** (1xxx, e.g. /tmp) lets only the file owner delete files. Example: **chmod 1777 /shared**. Use **getfacl/setfacl** for fine-grained ACLs.

2.5 Processes and system monitoring

A **process** is a running program with a PID. Linux uses signals to control processes.

```
ps aux | grep nginx      # snapshot of processes
ps -ef --forest          # parent-child tree
top; htop                # live view (htop is friendlier); press M=memory sort, P=cpu
pgrep -a java            # find PIDs by name
kill 1234                # SIGTERM (15): polite shutdown
kill -9 1234             # SIGKILL: force, no cleanup (last resort)
kill -HUP 1234           # SIGHUP: many daemons reload config
pkill -f "python app.py" # kill by pattern; killall nginx
nohup ./job.sh > job.log 2>&1 & # keep running after logout
jobs; fg %1; bg %1       # job control in current shell
nice -n 10 ./heavy.sh    # start with lower priority; renice -n 5 -p 1234

free -h                 # memory and swap
uptime                  # load averages (1, 5, 15 min)
vmstat 2 5              # cpu, mem, io every 2s, 5 times
iostat -xz 2            # disk IO per device (sysstat package)
lsof -i :8080           # which process uses port 8080
lsof -p 1234            # files opened by a process
dmesg -T | tail         # kernel messages (OOM kills, disk errors)
```

Load average is the average number of runnable (and uninterruptible) tasks. A load of 4 on a 4-core machine means fully busy; sustained load well above core count means saturation. **nproc** shows the core count.

Zombie and orphan processes

A **zombie** has finished but its parent has not read its exit status (shows **Z** in ps); fix by fixing/restarting the parent. An **orphan** has lost its parent and is adopted by PID 1 (init/systemd).

2.6 Services with systemd

systemd starts, stops and supervises services on modern distributions.

```
sudo systemctl status nginx          # state, recent logs
sudo systemctl start|stop|restart|reload nginx
sudo systemctl enable --now nginx    # start at boot AND now
sudo systemctl disable nginx
systemctl list-units --type=service --state=failed
systemctl daemon-reload              # after editing unit files
journalctl -u nginx -f               # follow logs of a service
journalctl -u nginx --since "1 hour ago" -p err
journalctl -b -1                     # logs from previous boot
```

A minimal custom service unit `/etc/systemd/system/myapp.service`:

```
[Unit]
Description=My Node App
After=network.target

[Service]
User=deploy
WorkingDirectory=/opt/myapp
ExecStart=/usr/bin/node server.js
Restart=on-failure
RestartSec=5
Environment=NODE_ENV=production

[Install]
WantedBy=multi-user.target
```

Then: `sudo systemctl daemon-reload && sudo systemctl enable --now myapp`.

2.7 Package management

Task	Debian / Ubuntu	RHEL / Amazon Linux / Fedora	Alpine
Refresh index	<code>apt update</code>	<code>dnf makecache</code>	<code>apk update</code>
Install	<code>apt install -y nginx</code>	<code>dnf install -y nginx</code>	<code>apk add nginx</code>
Upgrade all	<code>apt upgrade -y</code>	<code>dnf upgrade -y</code>	<code>apk upgrade</code>
Remove	<code>apt remove nginx</code>	<code>dnf remove nginx</code>	<code>apk del nginx</code>
Search	<code>apt search word</code>	<code>dnf search word</code>	<code>apk search word</code>
Which package owns file	<code>dpkg -S /path</code>	<code>rpm -qf /path</code>	<code>apk info -W /path</code>
List installed	<code>dpkg -l</code>	<code>rpm -qa</code>	<code>apk info</code>

2.8 Networking commands

```
ip addr show; ip -br a              # interfaces and IPs (replaces ifconfig)
ip route; ip route get 8.8.8.8
ping -c 4 example.com               # reachability
traceroute example.com; mtr example.com # path and latency per hop
ss -tulpn                           # listening TCP/UDP sockets with process (replaces netstat)
ss -s                               # socket summary
curl -I https://example.com          # headers only
curl -sS -X POST -H "Content-Type: application/json" -d '{"a":1}' http://localhost:8080/api
curl -o file.zip -L https://host/file.zip # download, follow redirects
wget -c https://host/file.iso         # resumable download
dig example.com +short; dig MX example.com # DNS lookup; nslookup also works
nc -zv host 5432                      # test whether a TCP port is open
sudo tcpdump -i eth0 port 80 -n -c 20  # capture packets
cat /etc/resolv.conf /etc/hosts        # DNS resolver and static host entries
hostname -I                           # IP addresses of this host
```

Firewall basics

```
# Ubuntu: ufw (simple front end)
sudo ufw allow 22/tcp; sudo ufw allow 80,443/tcp; sudo ufw enable; sudo ufw status verbose
# RHEL family: firewallld
sudo firewall-cmd --permanent --add-service=http && sudo firewall-cmd --reload
# Underlying: iptables / nftables
sudo iptables -L -n -v
```

2.9 SSH and secure remote access

SSH uses public/private key pairs. The **private key never leaves your machine**; the **public key** goes into `~/.ssh/authorized_keys` on the server.

```
ssh-keygen -t ed25519 -C "me@company.com"      # generate key pair (~/.ssh/id_ed25519)
ssh-copy-id user@server                        # install public key on server
ssh user@server                               # connect
ssh -i ~/.ssh/key.pem ec2-user@1.2.3.4        # with specific key (AWS style)
scp file.txt user@server:/tmp/                # copy file; scp -r dir/ ...
rsync -avz --delete ./site/ user@server:/var/www/site/ # efficient sync
ssh -L 5432:db.internal:5432 user@bastion     # local port forward (tunnel to private DB)
ssh -J bastion user@private-host              # jump host (ProxyJump)
chmod 600 ~/.ssh/id_ed25519; chmod 700 ~/.ssh  # required permissions or ssh refuses
```

`~/.ssh/config` makes this easy:

```
Host prod-web
  HostName 10.0.1.25
  User ubuntu
  IdentityFile ~/.ssh/prod.pem
  ProxyJump bastion
```

Hardening (`/etc/ssh/sshd_config`): `PermitRootLogin no`, `PasswordAuthentication no`, `AllowUsers deploy`, then `sudo systemctl reload sshd`. Add fail2ban to block brute force attempts.

2.10 Disks, mounts and archives

```
lsblk -f                                     # block devices and filesystems
sudo fdisk -l                               # partition tables
sudo mkfs.ext4 /dev/xvdf                    # create filesystem (DESTROYS data)
sudo mkdir /data && sudo mount /dev/xvdf /data
blkid                                       # get UUID, then add to /etc/fstab for persistent mounts:
# UUID=xxxx-xxxx /data ext4 defaults,nofail 0 2
sudo mount -a                               # test fstab entries before reboot!
# archives
tar -czvf backup.tar.gz /opt/app           # create gzip tar
tar -xzf backup.tar.gz -C /restore         # extract to directory
zip -r site.zip site/; unzip site.zip
```

LVM (advanced) lets you resize volumes: physical volume (`pvcreate`) -> volume group (`vgcreate`) -> logical volume (`lvcreate`) -> `lvextend -r -L +10G /dev/vg/lv`.

2.11 Shell scripting

A script is a text file of commands. Start with a shebang and use strict mode.

```
#!/usr/bin/env bash
set -euo pipefail          # exit on error, undefined var, and pipe failure
IFS=$'\n\t'

APP_DIR="${1:-/opt/app}"  # first argument with default
LOG="/var/log/deploy.log"

log() { echo "$(date '+%F %T') [INFO] $" | tee -a "$LOG"; }

backup() {
    local ts; ts=$(date +%Y%m%d_%H%M%S)
    tar -czf "/backup/app_${ts}.tar.gz" "$APP_DIR"
    log "backup created app_${ts}.tar.gz"
}

if [[ ! -d "$APP_DIR" ]]; then
    echo "Directory $APP_DIR missing" >&2
    exit 1
fi

for svc in nginx myapp; do
    if systemctl is-active --quiet "$svc"; then
        log "$svc running"
    else
        log "$svc DOWN, restarting"; sudo systemctl restart "$svc"
    fi
done

backup
trap 'log "script exited with code $" EXIT'
```

Syntax cheat sheet

Feature	Example
Variables	<code>name="dev"; echo "\$name"</code> (always quote)
Arithmetic	<code>n=\$((5 + 3))</code>
Conditionals	<code>if [[-f file && \$x -gt 3]]; then ... elif ...; else ...; fi</code>
File tests	<code>-f file, -d dir, -e exists, -s non-empty, -x executable</code>
Number compare	<code>-eq -ne -lt -le -gt -ge</code>
Loop over list	<code>for f in *.log; do gzip "\$f"; done</code>
While read lines	<code>while read -r line; do echo "\$line"; done < hosts.txt</code>
Arrays	<code>arr=(a b c); echo "\${arr[1]}"; echo "\${#arr[@]}"</code>
Exit status	<code>\$?</code> last status; 0 means success
Arguments	<code>\$0</code> script, <code>\$1..\$9</code> , <code>\$@</code> all args, <code>\$#</code> count
Case	<code>case "\$env" in dev) ... ;; prod) ... ;; *) ... ;; esac</code>

Cron: scheduled jobs

```
crontab -e          # edit your jobs; crontab -l to list
# m h dom mon dow  command
0 2 * * * /opt/scripts/backup.sh >> /var/log/backup.log 2>&1 # daily 02:00
*/5 * * * * /opt/scripts/healthcheck.sh # every 5 minutes
30 1 * * 1-5 /opt/scripts/report.sh # weekdays 01:30
```

2.12 Troubleshooting playbook

Symptom	First commands to run	
Server slow	<code>uptime, top/htop, free -h, iostat -xz 2, dmesg -T</code>	
Disk full	<code>df -h, `du -h --max-depth=1 /`</code>	<code>sort -h, df -i (inodes), lsof +L1` (deleted but open files)</code>
Service will not start	<code>systemctl status X, journalctl -u X -n 100, check config syntax (nginx -t)</code>	
Cannot connect to port	<code>ss -tulpn, firewall rules, security group, nc -zv, curl -v</code>	
High memory	<code>`ps aux --sort=-%mem`</code>	<code>head, free -h, check swap use, OOM entries in dmesg`</code>
DNS issues	<code>dig host, /etc/resolv.conf, resolvectl status</code>	
Permission denied	<code>ls -l, id, parent directory execute bit, SELinux (getenforce, ausearch -m avc)</code>	

Common interview questions

- Difference between hard link and soft link? A hard link is another name for the same inode (cannot cross filesystems or point to directories); a symlink is a separate file holding a path.
- What does `kill -9` do vs `kill`? SIGKILL cannot be caught or cleaned up; SIGTERM allows graceful shutdown.
- What is an inode? Metadata structure for a file (owner, permissions, block locations); filenames map to inodes. You can run out of inodes even with free space.
- What is the difference between `&&`, `;` and `||`? Sequential regardless; run next only on success; run next only on failure.
- How do you find which process holds a port? `ss -tulpn | grep :8080` or `lsof -i :8080`.

3. Git and Version Control

3.1 Concepts

Git is a **distributed version control system**: every clone holds the full history. It stores snapshots of your project as **commits**, each identified by a SHA hash and pointing to its parent(s). A **branch** is just a movable pointer to a commit; **HEAD** points to the branch (or commit) you are on.

The three areas

Area	What it is	Moves with
Working directory	Your actual files on disk	edit files
Staging area (index)	Snapshot being prepared for the next commit	<code>git add</code>
Repository (.git)	Committed history	<code>git commit</code>
Remote	Shared repo (GitHub, GitLab, Bitbucket)	<code>git push</code> / <code>git fetch</code> / <code>git pull</code>

3.2 Daily workflow commands

```
git config --global user.name "Your Name"
git config --global user.email "you@company.com"
git config --global init.defaultBranch main
git config --global pull.rebase true          # linear history on pull

git init                                     # new repo here
git clone https://github.com/org/repo.git    # copy a remote repo
git status -sb                              # short status with branch
git add file.txt                            # stage one file; git add -p (stage hunks interactively)
git add .                                    # stage everything
git commit -m "feat: add login endpoint"
git commit --amend --no-edit                # add staged changes to the last commit (local only!)
git log --oneline --graph --decorate --all  # compact history graph
git diff                                     # unstaged changes; git diff --staged for staged
git show a1b2c3d                            # details of a commit
git blame file.txt                          # who changed each line
```

Write commit messages in the imperative mood and keep the subject under ~50 characters. A popular convention is **Conventional Commits**: `feat:`, `fix:`, `docs:`, `refactor:`, `test:`, `chore:`, which also allows automated changelogs and version bumps.

3.3 Branching and merging

```
git branch                                # list local branches; -a includes remotes
git switch -c feature/payment             # create and switch (older: git checkout -b)
git switch main
git merge feature/payment                 # merge branch into current branch
git branch -d feature/payment             # delete merged branch; -D to force
git push -u origin feature/payment        # publish and set upstream
git push origin --delete feature/payment
```

Merge vs rebase

	Merge	Rebase
Result	Creates a merge commit; history is preserved as it happened	Replays your commits on top of the target; linear history
Safe on shared branches	Yes	No (rewrites commits; never rebase commits others have pulled)
Conflicts	Resolve once	May resolve per replayed commit

	Merge	Rebase
Use for	Merging features into main (with PRs)	Cleaning up your *local* feature branch before a PR

```
git switch feature/x
git rebase main           # replay my commits on the latest main
git rebase -i HEAD~4      # interactive: squash, reword, reorder, drop last 4 commits
git merge --squash feature/x # combine whole branch into one commit
git merge --no-ff feature/x # always create a merge commit
```

Resolving conflicts

When two branches change the same lines, Git marks the file:

```
<<<<<< HEAD
timeout = 30
=====
timeout = 60
>>>>>> feature/x
```

Edit the file to the desired result, remove the markers, then `git add file` and `git commit` (merge) or `git rebase --continue`. Abort with `git merge --abort` or `git rebase --abort`.

3.4 Undoing things safely

Goal	Command	Notes
Discard unstaged edits in a file	<code>git restore file</code>	Cannot be recovered
Unstage a file	<code>git restore --staged file</code>	Keeps the edits
Undo last commit, keep changes staged	<code>git reset --soft HEAD~1</code>	History rewritten locally
Undo last commit, keep changes unstaged	<code>git reset --mixed HEAD~1</code>	Default mode
Undo last commit and throw changes away	<code>git reset --hard HEAD~1</code>	Dangerous
Undo a pushed commit safely	<code>git revert <sha></code>	Creates a new inverse commit; safe for shared branches
Recover "lost" commits	<code>git reflog</code> then <code>git switch -c rescue <sha></code>	Reflog keeps ~90 days of HEAD movement

```
git stash push -m "wip login" # shelve uncommitted work
git stash list; git stash pop  # restore and drop; git stash apply keeps it
git cherry-pick a1b2c3d        # copy one commit onto current branch
git clean -fd                  # remove untracked files/dirs (use -n for dry run first)
```

3.5 Remotes, tags and collaboration

```
git remote -v; git remote add upstream https://github.com/orig/repo.git
git fetch --all --prune # download refs without changing your files
git pull --rebase        # fetch + rebase local commits
git push --force-with-lease # safer force push (fails if remote has new commits)
git tag -a v1.2.0 -m "Release 1.2.0"; git push origin v1.2.0
git log v1.1.0..v1.2.0 --oneline # what changed between releases
```

Pull Request (GitHub) / Merge Request (GitLab): a request to merge a branch, with code review and automated CI checks. Branch protection rules can require passing checks and approvals before merging.

3.6 Branching strategies

Strategy	Idea	Best for
GitHub Flow	main is always deployable; short-lived feature branches + PR	Web apps, continuous deployment
Trunk-based development	Everyone commits to main (or tiny branches) daily; feature flags hide unfinished work	High-performing CI/CD teams
Git Flow	main , develop , feature/* , release/* , hotfix/*	Versioned/packaged releases, slower cadence
Environment branches	dev , staging , prod branches	Simple GitOps setups (but drifts easily)

3.7 Advanced Git

```
git bisect start; git bisect bad; git bisect good v1.0 # binary search for a bug-introducing commit
git worktree add ../hotfix main # second working dir for same repo
git submodule add https://github.com/org/lib.git libs/lib; git submodule update --init --recursive
git log -S"password" --oneline # find commits that added/removed a string
git shortlog -sn # commits per author
git config --global alias.lg "log --oneline --graph --decorate --all"
```

.gitignore and secrets

.gitignore lists files Git must not track (build output, **node_modules/**, **.env**, ***.tfstate**, ***.pem**). If a secret is committed, **rotating the secret is mandatory**; removing it from history (**git filter-repo** or BFG) is secondary. Use pre-commit hooks or tools like **gitleaks** to block secrets.

Git hooks

Scripts in **.git/hooks/** run on events. Example **pre-commit** that runs tests:

```
#!/bin/sh
npm test || { echo "Tests failed, commit aborted"; exit 1; }
```

Server-side events are often handled with **webhooks**: GitHub notifies Jenkins on every push so it can start a build.

Interview questions

- **git fetch** vs **git pull**? Fetch downloads only; pull = fetch + merge (or rebase).
- **reset** vs **revert**? Reset moves the branch pointer (rewrites history); revert adds a new commit that undoes a previous one.
- What is detached HEAD? HEAD points at a commit rather than a branch; new commits are orphaned unless you create a branch.
- How do you remove a file from Git but keep it locally? **git rm --cached file** then commit and add to **.gitignore**.

4. Cloud Computing Fundamentals

4.1 What is cloud computing?

Cloud computing is on-demand access to computing resources (servers, storage, databases, networking, software) over the internet with **pay-as-you-go** pricing. Instead of buying hardware, you rent exactly what you need and scale it up and down in minutes.

Five essential characteristics

On-demand self-service, broad network access, resource pooling (multi-tenancy), rapid elasticity, and measured service (metered billing).

Service models

Model	You manage	Provider manages	Examples
IaaS (Infrastructure)	OS, runtime, app, data	Hardware, virtualization, network	EC2, Azure VM, Compute Engine
PaaS (Platform)	App and data	OS, runtime, scaling	Elastic Beanstalk, App Service, App Engine
CaaS (Containers)	Containers and config	Orchestrator control plane	EKS, AKS, GKE, ECS
FaaS / Serverless	Function code only	Everything else, per-invocation billing	Lambda, Azure Functions, Cloud Functions
SaaS (Software)	Just use it	Entire stack	Gmail, Office 365, Salesforce

Deployment models

Public (shared provider infrastructure), **Private** (dedicated to one organization, e.g. OpenStack/VMware), **Hybrid** (on-prem + public connected by VPN/Direct Connect), **Multi-cloud** (more than one public provider to avoid lock-in or use best services).

Shared responsibility model

The provider secures the cloud **itself** (data centers, hardware, hypervisor); you secure what you put **in** the cloud (data, IAM, OS patches on IaaS, network rules, encryption choices).

4.2 Global infrastructure

- **Region**: a geographic area with multiple data centers (e.g. **ap-south-1** Mumbai).
- **Availability Zone (AZ)**: one or more isolated data centers inside a region. Spread workloads across 2+ AZs for high availability.
- **Edge locations / CDN**: caches content close to users (CloudFront, Azure Front Door, Cloud CDN).
- Choose a region by latency, data residency law, service availability and price.

4.3 Core services mapped across providers

Category	AWS	Azure	GCP
Virtual machines	EC2	Virtual Machines	Compute Engine
Object storage	S3	Blob Storage	Cloud Storage
Block storage	EBS	Managed Disks	Persistent Disk
File storage	EFS	Azure Files	Filestore
Virtual network	VPC	VNet	VPC
Load balancer	ELB (ALB/NLB)	Load Balancer / App Gateway	Cloud Load Balancing
Identity	IAM	Entra ID (Azure AD) + RBAC	Cloud IAM
Managed Kubernetes	EKS	AKS	GKE
Container service	ECS / Fargate	Container Apps / ACI	Cloud Run
Serverless functions	Lambda	Functions	Cloud Functions
Relational DB	RDS / Aurora	Azure SQL / Database for PostgreSQL	Cloud SQL / AlloyDB
NoSQL	DynamoDB	Cosmos DB	Firestore / Bigtable
DNS	Route 53	Azure DNS	Cloud DNS
Monitoring	CloudWatch	Azure Monitor	Cloud Monitoring
IaC native	CloudFormation	ARM / Bicep	Deployment Manager
Secrets	Secrets Manager / SSM Parameter Store	Key Vault	Secret Manager
CI/CD native	CodePipeline / CodeBuild	Azure DevOps	Cloud Build

4.4 Compute in depth

Virtual machines (EC2 example)

An instance is created from an **image** (AMI) with an **instance type** (vCPU + RAM + network), a **key pair** for SSH, a **security group**, and storage volumes.

Instance family (AWS)	Optimized for	Example
General purpose	Balanced	t3, m6i
Compute optimized	CPU-bound	c6i
Memory optimized	In-memory DBs	r6i
Storage optimized	High IOPS	i3
Accelerated	GPU / ML	p4, g5

Pricing models: On-Demand (pay per second), Reserved/Savings Plans (1 to 3 year commitment, up to ~70% off), Spot (spare capacity, up to ~90% off but can be interrupted; great for CI agents and batch jobs).

Auto Scaling: a group of identical instances behind a load balancer that grows or shrinks based on metrics (CPU, request count) or schedules, and replaces unhealthy instances automatically. **Scale up/vertical** = bigger machine; **scale out/horizontal** = more machines (preferred in cloud).

Serverless

You upload a function; the platform runs it on events (HTTP, queue message, file upload) and bills per request and duration. Pros: no server management, automatic scaling to zero. Cons: cold starts, execution time limits, harder local debugging, vendor lock-in.

4.5 Networking: VPC

A **VPC** is your private, isolated network in the cloud.

- **CIDR block:** IP range, e.g. **10.0.0.0/16** (65,536 addresses). **/24** = 256 addresses.
- **Subnet:** slice of the VPC in one AZ. **Public subnet** has a route to an **Internet Gateway (IGW)**; **private subnet** has no direct route, outbound internet goes via a **NAT Gateway** placed in a public subnet.
- **Route table:** rules saying where traffic for a CIDR goes (IGW, NAT, peering, VPN, Transit Gateway).
- **Security Group (SG):** **stateful** virtual firewall attached to an instance/ENI; allow rules only; return traffic is automatically allowed.
- **Network ACL (NACL):** **stateless** subnet-level firewall; allow and deny rules, evaluated in order; return traffic must be explicitly allowed.
- **VPC peering / Transit Gateway / VPN / Direct Connect:** ways to connect VPCs and on-premises networks.
- **Bastion host / SSM Session Manager:** controlled way into private instances.

A standard three-tier layout: public subnets (load balancer, NAT), private app subnets (EC2/containers), private data subnets (databases), replicated across at least two AZs.

Security Group	Network ACL
Instance level	Subnet level
Stateful	Stateless
Allow rules only	Allow and deny
All rules evaluated	Rules evaluated in number order

4.6 Storage

Type	Description	Use case
Object (S3)	Flat key-value of objects with metadata, HTTP API, virtually unlimited, 11 nines durability	Backups, static sites, logs, artifacts, data lakes
Block (EBS)	Raw disk attached to one instance	OS disks, databases
File (EFS/NFS)	Shared POSIX filesystem mounted by many instances	Shared content, CMS

S3 storage classes (Standard, Infrequent Access, Glacier tiers) plus **lifecycle rules** move data to cheaper tiers automatically. Enable **versioning** and **encryption**, and block public access by default.

4.7 Identity and Access Management (IAM)

IAM answers: **who** can do **what** on **which** resource.

- **User:** a person or app with long-lived credentials (prefer SSO and roles instead).
- **Group:** collection of users sharing policies.
- **Role:** an identity assumed temporarily by a user, service or another account, with short-lived credentials. EC2 instances and Lambda use roles instead of stored keys.
- **Policy:** JSON document of permissions.

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": ["s3:GetObject", "s3:PutObject"],
    "Resource": "arn:aws:s3:::my-app-bucket/*"
  }]
}
```

Best practices: least privilege, MFA on all human accounts (especially root), no root usage, no access keys in code, rotate credentials, use roles for workloads, enable CloudTrail (audit log of API calls).

4.8 Databases and load balancing

- **Relational (RDS):** managed MySQL/PostgreSQL/SQL Server; automated backups, patching, **Multi-AZ** standby for failover, **read replicas** for read scaling.
- **NoSQL (DynamoDB):** key-value/document, single digit ms latency at any scale, pay per request or provisioned capacity.
- **Caching (ElastiCache/Redis):** reduce DB load and latency.
- **Load balancer types:** **ALB** (layer 7, HTTP routing by path or host, WebSockets), **NLB** (layer 4, TCP/UDP, extreme performance, static IP), **GWL** (appliances). Health checks remove failed targets from rotation.

4.9 Well-Architected pillars

Operational excellence, Security, Reliability, Performance efficiency, Cost optimization, Sustainability. Key reliability terms: **High availability** (survive component failure, multi-AZ), **Fault tolerance**, **Disaster recovery** with **RPO** (how much data loss is acceptable) and **RTO** (how long recovery may take). DR strategies from cheap/slow to expensive/fast: Backup and restore, Pilot light, Warm standby, Multi-site active/active.

4.10 AWS CLI cheat sheet

```
aws configure                                # set access key, secret, region (or use SS0: aws configure s \
so)
aws sts get-caller-identity                  # who am I?
aws ec2 describe-instances --query "Reservations[].Instances[].[InstanceId,State.Name,PublicIpAddress]" \
--output table
aws ec2 run-instances --image-id ami-0abc --instance-type t3.micro --key-name mykey \
--security-group-ids sg-123 --subnet-id subnet-456 --count 1
aws ec2 stop-instances --instance-ids i-0123456789abcdef0
aws s3 ls; aws s3 mb s3://my-unique-bucket
aws s3 cp app.zip s3://my-unique-bucket/releases/
aws s3 sync ./dist s3://my-site-bucket --delete
aws iam list-users; aws iam create-role --role-name x --assume-role-policy-document file://trust.json
aws eks update-kubeconfig --name prod-cluster --region ap-south-1 # connect kubectl to EKS
aws logs tail /aws/lambda/myfn --follow
aws ecr get-login-password | docker login --username AWS --password-stdin 123456789012.dkr.ecr.ap-south- \
1.amazonaws.com
```

Azure and GCP equivalents: `az login`, `az vm list -o table`, `az aks get-credentials`; `gcloud auth login`, `gcloud compute instances list`, `gcloud container clusters get-credentials`.

4.11 Cost management tips

Tag everything (`env`, `team`, `owner`), set budgets and alerts, right-size instances, shut down dev environments at night, use Spot for fault-tolerant work, delete unattached volumes and old snapshots, and use lifecycle policies on storage. Review bills monthly (Cost Explorer).

Interview questions

- Region vs AZ? Region = geographic area; AZ = isolated data center(s) within it.
- Security group vs NACL? Stateful instance firewall vs stateless subnet firewall.
- How does a private instance reach the internet? Through a NAT Gateway in a public subnet.
- IAM user vs role? Users have long-term credentials; roles provide temporary credentials and are assumed.
- How do you make an app highly available on AWS? Multi-AZ ASG behind an ALB, Multi-AZ database, stateless app tier, health checks, backups.